SEMESTER 3 • PROGRAM CORE • DETAILED BILINGUAL HANDBOOK

Programming in C

A complete syllabus-based handbook for Computer Engineering and Computer Hardware Engineering students. It develops practical programming ability through control structures, functions, recursion, arrays, searching, sorting, strings, pointers, dynamic memory, structures, unions, files and command-line arguments.

COURSE CODE
3132

SEMESTER
3

CREDITS
3

CATEGORY
Program Core

PERIODS / WEEK
3 (L:3, T:0, P:0)

PERIODS / SEMESTER
45

INSTRUCTION
43 hours

SERIES TESTS
2 hours

45

Total periods

Four course outcomes, thirty syllabus sub-outcomes and two series tests.

COURSE PURPOSE

What you should be able to do

Write structured programs

Convert a problem into input, processing and output steps using sequence, selection, repetition and functions.

ഒരു പ്രശ്നത്തെ input → processing → output എന്ന ഘടനയിലാക്കി sequence, selection, loop, function എന്നിവ ഉപയോഗിച്ച് program എഴുതുക.

Process collections of data

Use arrays, searching, sorting and strings to solve realistic problems.

Arrays, searching, sorting, strings എന്നിവ ഉപയോഗിച്ച് data കാര്യക്ഷമമായി process ചെയ്യുക.

Use memory efficiently

Apply pointers, pointer arithmetic and dynamic memory safely.

Pointer, malloc, calloc, realloc, free എന്നിവ സുരക്ഷിതമായി ഉപയോഗിക്കുക.

Model records

Create user-defined data types using structures, unions and enumerations.

വ്യത്യസ്ത data types ഒന്നിച്ച് record ആയി സൂക്ഷിക്കാൻ structure ഉപയോഗിക്കുക.

Store data permanently

Read and write sequential files with error checking.

Files തുറക്കുമ്പോൾ result പരിശോധിച്ച് അവസാനം fclose വിളിക്കുക.

Build a foundation

Prepare for data structures, system programming and embedded software.

Advanced programming subjects-ന് ആവശ്യമായ foundation നേടുക.

SYLLABUS STRUCTURE

Course-outcome and hour map

The four CO durations total 43 teaching hours. Two one-hour series tests make the stated 45 periods.

CO1 / Module 1	**11 hours**
CO2 / Module 2	**11 hours + Test I**
CO3 / Module 3	**10 hours**
CO4 / Module 4	**11 hours + Test II**

CO	Outcome	Hours	Main content
CO1	Use sequential, conditional, looping structures and functions.	11	Control structures, preprocessor, functions, storage classes and recursion.
CO2	Use arrays to solve real-world problems.	11	Arrays, searching, sorting, strings and array parameters.
CO3	Develop programs using pointers.	10	Pointer operations, dynamic memory and arrays through pointers.
CO4	Construct user-defined data types and use files.	11	Structures, union, enum, files and command-line arguments.

Prerequisite: Semester 2 Problem Solving and Programming. All four COs are strongly mapped to PO1.

SETUP AND GOOD PRACTICE

Compile correctly before studying programs

Recommended GCC command

Terminal

```
gcc -std=c11 -Wall -Wextra -pedantic program.c -o program
```

Run with `program.exe` on Windows or `./program` on Linux/macOS.

Warnings ignore ചെയ്യരുത്. അവ compile-time mistakes കണ്ടെത്താൻ സഹായിക്കും.

Use standard C

- ✓ Use `int main(void)`.
- ✓ Return 0 on success.
- ✓ Avoid non-standard conio.h functions.
- ✓ Never use unsafe gets().

CO1 • 11 HOURS

Module 1 — Control Structures, Functions, Storage Classes and Recursion

Use the basic building blocks of C to create modular and maintainable programs.

M1.01

2 hours • Understanding

C program structure and control flow

A C program normally contains preprocessor directives, declarations, `main()`, statements and user-defined functions. Execution begins at `main()`.

Sequence **if / else** **switch** **for** **while** **do-while** **break** **continue** **goto** **return**

മലയാളം: Sequence-ൽ statements ക്രമത്തിൽ നടക്കും. Selection-ൽ condition അനുസരിച്ച് branch തിരഞ്ഞെടുക്കും. Loop-ൽ ഒരു block ആവർത്തിക്കും. `break` loop/switch അവസാനിപ്പിക്കും; `continue` അടുത്ത iteration-ലേക്ക് പോകും.

Example — largest of three numbers

```
#include <stdio.h>

int main(void)
{
    int a, b, c, largest;

    printf("Enter three integers: ");
    if (scanf("%d %d %d", &a, &b, &c) != 3) {
        fprintf(stderr, "Invalid input\n");
        return 1;
    }

    largest = a;
    if (b > largest) {
        largest = b;
    }
    if (c > largest) {
        largest = c;
    }

    printf("Largest = %d\n", largest);
    return 0;
}
```

Loop-selection guide

Construct	Use when	Key caution
for	Initialization, condition and update are naturally grouped; count-controlled loops.	Check off-by-one errors.
while	Number of repetitions is unknown and condition is checked first.	Update loop state to avoid an infinite loop.
do-while	Body must run at least once.	Semicolon is required after `while(condition);`.
switch	One expression is compared with discrete integral/enum cases.	Add `break` unless fall-through is intentional.

M1.02

1 hour

Preprocessor: file inclusion and macros

Preprocessor directives begin with `#` and are processed before compilation. `#include` inserts declarations from a header; `#define` performs macro substitution.

Safe macro style

```
#include <stdio.h>

#define PI 3.141592653589793
#define SQUARE(x) ((x) * (x))

int main(void)
{
    double radius = 3.0;
    printf("Area = %.2f\n", PI * SQUARE(radius));
    return 0;
}
```

Macro caution: Always parenthesize parameters and the complete replacement expression. Even then, avoid arguments with side effects such as `SQUARE(i++)`.

Macro text substitution ആണ്; function പോലെ type checking ഇല്ല. അതിനാൽ parentheses ഉപയോഗിക്കുക.

M1.03–M1.04

2 hours

Modular programming and functions

Break a large problem into small functions. A function has a declaration/prototype, definition and call. Parameters carry input; the return value carries one result.

Function prototype, definition and call

```
#include <stdio.h>

int gcd(int a, int b);

int main(void)
{
    printf("GCD = %d\n", gcd(48, 18));
    return 0;
}

int gcd(int a, int b)
{
    while (b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

Function ഒരു പ്രത്യേക task ചെയ്യണം. Meaningful name, ചെറിയ parameter list, clear return value എന്നിവ modularity കൂട്ടും.

M1.05–M1.06

3 hours

Storage classes, scope, lifetime and visibility

Storage class	Typical location	Lifetime	Visibility / purpose
auto	Inside block; default local variable	During block execution	Block scope.
register	Inside block	During block execution	Compiler hint for frequent access; address may not be taken.
static local	Inside function/block	Entire program	Block scope but retains value between calls.
static global	File scope	Entire program	Visible only in the current source file.
extern	File or block declaration	Entire program for the defined object	Refers to an object defined elsewhere.

Static local variable

```
#include <stdio.h>

void visit(void)
{
    static int count = 0;
    count++;
    printf("Function called %d time(s)\n", count);
}

int main(void)
{
    visit();
    visit();
    visit();
    return 0;
}
```

Scope variable name എവിടെ ഉപയോഗിക്കാമെന്ന് പറയുന്നു. **Lifetime** memory-ൽ variable എത്രകാലം നിലനിൽക്കും എന്ന് പറയുന്നു. Static local variable function calls ഇടയിലും value സൂക്ഷിക്കും.

M1.07–M1.08

3 hours

Recursion

A recursive function calls itself directly or indirectly. Every correct recursive solution needs a **base case** that stops recursion and a **recursive case** that moves toward the base case.

Recursive factorial with validation

```
#include <stdio.h>

unsigned long long factorial(unsigned int n)
{
    if (n <= 1U) {
        return 1ULL;
    }
    return n * factorial(n - 1U);
}

int main(void)
{
    unsigned int n;

    printf("Enter n (0 to 20): ");
    if (scanf("%u", &n) != 1 || n > 20U) {
        fprintf(stderr, "Invalid range\n");
        return 1;
    }

    printf("%u! = %llu\n", n, factorial(n));
    return 0;
}
```

Base case

Returns without another recursive call.

Recursion നിർത്തുന്ന condition.

Progress

Each call reduces the problem size.

ഓരോ call-ലും base case-ലേക്ക് അടുത്തുവരണം.

Stack cost

Each call consumes stack memory; deep recursion can overflow.

വളരെ deep recursion stack overflow ഉണ്ടാക്കാം.

Module 1 exam focus: control-flow syntax, function prototype/definition/call, macro precautions, storage-class comparison, scope/lifetime and a traced recursive program.

CO2 • 11 HOURS + SERIES TEST I

Module 2 — Arrays, Searching, Sorting and Strings

Use indexed collections and algorithms to organise, locate, rearrange and process data.

M2.01–M2.02

2 hours

One-dimensional and multidimensional arrays

An array stores elements of the same type in contiguous memory. Indexing begins at zero. A two-dimensional array is commonly treated as rows and columns and is stored in row-major order in C.

Average and maximum using a 1D array

```
#include <stdio.h>

#define SIZE 5

int main(void)
{
    int values[SIZE] = {12, 7, 25, 9, 18};
    int sum = 0;
    int maximum = values[0];

    for (int i = 0; i < SIZE; i++) {
        sum += values[i];
        if (values[i] > maximum) {
            maximum = values[i];
        }
    }

    printf("Average = %.2f\n", (double)sum / SIZE);
    printf("Maximum = %d\n", maximum);
    return 0;
}
```

Matrix addition

```
#include <stdio.h>

#define ROWS 2
#define COLS 3

int main(void)
{
    int a[ROWS][COLS] = {{1, 2, 3}, {4, 5, 6}};
    int b[ROWS][COLS] = {{6, 5, 4}, {3, 2, 1}};
    int sum[ROWS][COLS];

    for (int row = 0; row < ROWS; row++) {
        for (int col = 0; col < COLS; col++) {
            sum[row][col] = a[row][col] + b[row][col];
            printf("%d%c", sum[row][col],
                   col == COLS - 1 ? '\n' : ' ');
        }
    }
    return 0;
}
```

Array size കടക്കുന്നത് undefined behaviour ആണ്. Valid index range `0` മുതൽ `size - 1` വരെ മാത്രം.

M2.03

1 hour

Divide and conquer

Divide and conquer splits a problem into smaller subproblems, solves them and combines the results. Binary search and quicksort are important syllabus examples.

1. Divide the input.
2. Conquer smaller parts recursively or iteratively.
3. Combine or interpret the results.

വലിയ problem ചെറിയ parts ആയി വിഭജിച്ച് solve ചെയ്യുന്നതാണ് divide and conquer. Binary search sorted array-നെ പകുതികളാക്കുന്നു.

M2.04

2 hours

Algorithm	Requirement	Worst time
Linear search	None	O(n)
Binary search	Array must be sorted	O(log n)

Binary search

```
int binary_search(const int a[], int size, int target)
{
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid] == target) {
            return mid;
        }
        if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

Binary search ഉപയോഗിക്കുന്നതിന് മുമ്പ് array sorted ആണെന്ന് ഉറപ്പാക്കണം. Found അല്ലെങ്കിൽ index; not found ആണെങ്കിൽ `-1` return ചെയ്യാം.

M2.04 Included

Selection sort and quicksort

Selection sort

Repeatedly finds the smallest remaining element and places it at the next sorted position. It is simple, in-place and $O(n^2)$.

Selection sort

```
void selection_sort(int a[], int size)
{
    for (int i = 0; i < size - 1; i++) {
        int min_index = i;

        for (int j = i + 1; j < size; j++) {
            if (a[j] < a[min_index]) {
                min_index = j;
            }
        }

        int temp = a[i];
        a[i] = a[min_index];
        a[min_index] = temp;
    }
}
```

Quicksort

Partitions the array around a pivot and recursively sorts the left and right parts. Average time is $O(n \log n)$; worst case is $O(n^2)$.

Quicksort

```
static void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

static int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quicksort(int a[], int low, int high)
{
    if (low < high) {
        int pivot_index = partition(a, low, high);
        quicksort(a, low, pivot_index - 1);
        quicksort(a, pivot_index + 1, high);
    }
}
```

Selection sort മനസ്സിലാക്കാൻ എളുപ്പമാണ്, പക്ഷേ large data-ക്ക് slow. Quicksort divide-and-conquer ഉപയോഗിക്കുന്നു; pivot partition ശരിയായി trace ചെയ്യുക.

M2.05–M2.06

4 hours

Strings in C

A C string is a character array terminated by the null character `'\0'`. Storage must include space for this terminator.

Operation	Library function	Header	Important note
Length	`strlen`	string.h	Does not count `'\0'`.
Copy	`strcpy`	string.h	Destination must be large enough.
Concatenate	`strcat`	string.h	Destination needs space for both strings and terminator.
Compare	`strcmp`	string.h	Returns 0 when equal.
Find substring	`strstr`	string.h	Returns pointer to first match or NULL.

Safe line input and custom string length

```
#include <stdio.h>
#include <string.h>

size_t my_strlen(const char text[])
{
    size_t length = 0;

    while (text[length] != '\0') {
        length++;
    }
    return length;
}

int main(void)
{
    char text[100];

    printf("Enter a line: ");
    if (fgets(text, sizeof text, stdin) == NULL) {
        fprintf(stderr, "Input error\n");
        return 1;
    }

    text[strcspn(text, "\n")] = '\0';
    printf("Length = %zu\n", my_strlen(text));
    return 0;
}
```

Do not use `gets()`: it cannot limit input length. Prefer `fgets()` and remove the trailing newline when needed.

String അവസാനിക്കുന്നതായി compiler അറിയുന്നത് null character `'\0'` കൊണ്ടാണ്. Character array size-ൽ terminator-നുള്ള സ്ഥലം വേണം.

Course 3132
M2.07

2 hours

Passing arrays to functions

When an array is passed to a function, the parameter behaves like a pointer to its first element. Pass the size separately because the function cannot automatically determine the full array length.

Array parameter with const

```
double average(const int values[], int size)
{
    int sum = 0;

    for (int i = 0; i < size; i++) {
        sum += values[i];
    }
    return size > 0 ? (double)sum / size : 0.0;
}
```

Function array മാറ്റേണ്ടതില്ലെങ്കിൽ parameter-ൽ `const` ഉപയോഗിക്കുക. Array size വേറെ argument ആയി pass ചെയ്യണം.

Series Test I focus: array initialization and traversal, matrix programs, linear/binary search, selection sort, quicksort, string representation and operations, and passing arrays to functions.

CO3 • 10 HOURS

Module 3 — Pointers and Dynamic Memory Allocation

Understand addresses and use pointers to share data, traverse arrays and allocate memory at run time.

M3.01

2 hours

Pointer fundamentals and operations

A pointer variable stores the address of another object. `&` obtains an address; unary `*` dereferences a valid pointer to access the pointed object.

Pointer declaration and dereferencing

```
#include <stdio.h>

int main(void)
{
    int value = 25;
    int *pointer = &value;

    printf("value = %d\n", value);
    printf("address = %p\n", (void *)pointer);
    printf("through pointer = %d\n", *pointer);

    *pointer = 40;
    printf("new value = %d\n", value);
    return 0;
}
```

Expression	Meaning
`int *p;`	Declares a pointer to int; it is not yet pointing to a valid int.
`p = &x;`	Stores the address of x.
`*p`	Accesses the int at the stored address.
`p + 1`	Moves to the next int object, not merely the next byte.
`p == NULL`	Checks that the pointer points to no object.

Uninitialized pointer dereference ചെയ്യരുത്. Valid object address അല്ലെങ്കിൽ allocated memory address മാത്രം pointer-ൽ സൂക്ഷിക്കുക.

M3.02 1 hour

Passing pointers to functions

Pointers let a function modify caller variables and return multiple results.

Swap using pointers

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* Call: swap(&x, &y); */
```

C arguments value ആയി pass ചെയ്യുന്നു. Address pass ചെയ്താൽ function ആ address dereference ചെയ്ത് original variable മാറ്റാം.

M3.03

1 hour

Dynamic memory allocation

Function	Purpose
malloc	Allocates uninitialized bytes.
calloc	Allocates an array and initializes bytes to zero.
realloc	Changes the size of an existing allocation.
free	Releases allocated memory.

Allocation failure ആണെങ്കിൽ NULL ലഭിക്കും. ഉപയോഗം കഴിഞ്ഞാൽ `free()` വിളിക്കണം; അതിനു ശേഷം pointer വീണ്ടും dereference ചെയ്യരുത്.

M3.03

Example

Dynamic array with safe reallocation

malloc + realloc + free

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count;

    printf("How many values? ");
    if (scanf("%d", &count) != 1 || count <= 0) {
        fprintf(stderr, "Invalid count\n");
        return 1;
    }

    int *values = malloc((size_t)count * sizeof *values);
    if (values == NULL) {
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }

    for (int i = 0; i < count; i++) {
        values[i] = (i + 1) * 10;
    }

    int new_count = count + 2;
    int *temporary = realloc(values,
                             (size_t)new_count * sizeof *values);
    if (temporary == NULL) {
        free(values);
        fprintf(stderr, "Reallocation failed\n");
        return 1;
    }
    values = temporary;

    values[count] = 100;
    values[count + 1] = 200;

    for (int i = 0; i < new_count; i++) {
        printf("%d%c", values[i],
               i == new_count - 1 ? '\n' : ' ');
    }

    free(values);
    values = NULL;
    return 0;
}
```

Correct realloc pattern: Store the result in a temporary pointer first. Assign it back only after checking for NULL, otherwise the original allocation could be lost.

M3.04–M3.05

6 hours

Arrays, strings and pointers

In most expressions, an array name is converted to a pointer to its first element. Therefore `a[i]` is equivalent to `*(a + i)`.

Traverse a 1D array with a pointer

```
int sum_array(const int *begin, int size)
{
    int sum = 0;

    for (int i = 0; i < size; i++) {
        sum += *(begin + i);
    }
    return sum;
}
```

Access a 2D array using pointer notation

```
#include <stdio.h>

int main(void)
{
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 3; col++) {
            printf("%d%c", *(*(matrix + row) + col),
                   col == 2 ? '\n' : ' ');
        }
    }
    return 0;
}
```

Array of pointers to strings

```
#include <stdio.h>

int main(void)
{
    const char *subjects[] = {
        "C Programming",
        "Database Management",
        "Digital Fundamentals"
    };

    int count = (int)(sizeof subjects / sizeof subjects[0]);

    for (int i = 0; i < count; i++) {
        printf("%s\n", subjects[i]);
    }
    return 0;
}
```

Array name സാധാരണ first element-ന്റെ address ആയി decay ചെയ്യും. എന്നാൽ `sizeof array` same scope-ൽ മുഴുവൻ array size നൽകുന്നു; function parameter-ൽ അത് pointer size മാത്രമായിരിക്കും.

Module 3 exam focus: address and dereference operators, pointer arithmetic, swap via pointers, NULL checks, malloc/calloc/realloc/free, array–pointer equivalence, strings through pointers and array of pointers.

CO4 • 11 HOURS + SERIES TEST II

Module 4 — Structures, Unions, Enumerations, Files and Command-Line Arguments

Represent real records and preserve program data using user-defined types and sequential files.

M4.01–M4.05

6 hours

Structures and structure processing

A structure groups related members that may have different types. Use the dot operator for an object and the arrow operator for a pointer to a structure.

Student record, array of structures and function parameter

```c
#include <stdio.h>

#define NAME_SIZE 40

struct Student {
    int roll;
    char name[NAME_SIZE];
    float mark;
};

void print_student(const struct Student *student)
{
    printf("%d | %s | %.1f\n",
           student->roll,
           student->name,
           student->mark);
}

int main(void)
{
    struct Student students[] = {
        {1, "Anu", 86.5f},
        {2, "Basil", 74.0f},
        {3, "Chitra", 91.0f}
    };

    int count = (int)(sizeof students / sizeof students[0]);

    for (int i = 0; i < count; i++) {
        print_student(&students[i]);
    }
    return 0;
}
```

Form	Example	Meaning
Structure definition	`struct Student { ... };`	Creates a structure type tag.
Object	`struct Student s;`	Creates one record.
Member access	`s.mark`	Accesses a member through an object.
Pointer member	`p->mark`	Equivalent to `(*p).mark`.
Array	`struct Student class[60];`	Stores many records.

Structure ഓരോ member-നും വേറെ storage നൽകുന്നു. Function-ലേക്ക് വലിയ structure copy ചെയ്യുന്നതിന് പകരം `const struct Student *` pass ചെയ്യുന്നത് efficient ആണ്.

M4.06

1 hour

Union

All union members share the same memory. At a given time, only the most recently written member should normally be read.

Tagged union concept

```
enum ValueType { INTEGER_VALUE, REAL_VALUE };

struct Value {
    enum ValueType type;
    union {
        int integer;
        double real;
    } data;
};
```

Union members same memory share ചെയ്യും. ഏത് member valid ആണെന്ന് സൂചിപ്പിക്കാൻ enum tag ഉപയോഗിക്കുന്നത് നല്ല design ആണ്.

M4.06

Included

Enumeration

An enumeration creates named integral constants, improving readability and restricting states conceptually.

Enum example

```
enum MenuChoice {
    ADD = 1,
    SEARCH,
    DISPLAY,
    EXIT
};
```

Magic numbers-ന് പകരം meaningful names നൽകാൻ enum ഉപയോഗിക്കുക.

Sequential files

A file pointer of type `FILE *` represents an open stream. Always check whether `fopen()` succeeded and close the file after use.

Mode	Meaning	Existing content
r	Read	File must exist.
w	Write	Truncated; new file created if needed.
a	Append	Writes at end; new file created if needed.
r+	Read and write	File must exist.
w+	Read and write	Truncated.
a+	Read and append	Writes at end.

Write and read student records in a text file

```
#include <stdio.h>

int main(void)
{
    FILE *file = fopen("marks.txt", "w");
    if (file == NULL) {
        perror("marks.txt");
        return 1;
    }

    fprintf(file, "%d %s %.1f\n", 1, "Anu", 86.5);
    fprintf(file, "%d %s %.1f\n", 2, "Basil", 74.0);

    if (fclose(file) != 0) {
        perror("fclose");
        return 1;
    }

    file = fopen("marks.txt", "r");
    if (file == NULL) {
        perror("marks.txt");
        return 1;
    }

    int roll;
    char name[40];
    float mark;

    while (fscanf(file, "%d %39s %f",
                  &roll, name, &mark) == 3) {
        printf("%d %s %.1f\n", roll, name, mark);
    }

    fclose(file);
    return 0;
}
```

Do not use `while (!feof(file))` as the input loop: test the return value of the actual input function, as shown above.

`fopen()` NULL ആണോ പരിശോധിക്കുക. Read loop-ൽ `fscanf` / `fgets` return value test ചെയ്യണം. അവസാനം `fclose()` വിളിക്കുക.

1 hour

Command-line arguments

`argc` is the number of command-line strings, including the program name. `argv` is an array of pointers to those strings.

Sum integers supplied on the command line

```
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    long total = 0;

    if (argc < 2) {
        fprintf(stderr, "Usage: %s number...\n", argv[0]);
        return 1;
    }

    for (int i = 1; i < argc; i++) {
        char *end;
        errno = 0;
        long value = strtol(argv[i], &end, 10);

        if (errno != 0 || *end != '\0') {
            fprintf(stderr, "Invalid integer: %s\n", argv[i]);
            return 1;
        }
        total += value;
    }

    printf("Total = %ld\n", total);
    return 0;
}
```

```
Example:
program 10 20 30
Total = 60
```

`argv[0]` program name ആണ്. User arguments `argv[1]` മുതൽ തുടങ്ങും. Text-to-number conversion validate ചെയ്യണം.

Series Test II focus: structure definition and processing, array of structures, passing structures and structure pointers, union versus structure, enum, file modes and functions, sequential file programs and command-line arguments.

PROGRAMMING PRACTICE

Recommended program laboratory

Write, compile, test, trace and improve every program. The objective is to understand the algorithm and memory behaviour—not merely memorise source code.

Control flow

1. Largest of three
2. Grade classification
3. Calculator using switch
4. Prime-number test
5. Armstrong number
6. Pattern printing

Functions

1. GCD and LCM
2. Function-based menu
3. Static call counter
4. Recursive factorial
5. Recursive Fibonacci
6. Recursive digit sum

Arrays

1. Average and maximum
2. Second largest
3. Matrix addition
4. Matrix multiplication
5. Linear/binary search
6. Selection sort

Strings

1. Length without strlen
2. Copy without strcpy
3. Concatenate
4. Palindrome
5. Word count
6. Substring search

Pointers

1. Swap
2. Array sum by pointer
3. Reverse array
4. String traversal
5. Dynamic array
6. Resize using realloc

Structures

1. Student record
2. Array of students
3. Highest scorer
4. Sort records
5. Pointer to structure
6. Tagged union

Files

1. Write/read marks
2. Character/word/line count
3. File copy
4. Append record
5. Search record
6. Update report file

Command line

1. Sum numbers
2. Display arguments
3. Copy file by names
4. Find maximum
5. Validate options

6. Count words in a file

Testing checklist: Test normal input, boundary values, invalid input, zero and negative values, empty data, one-element data, duplicates, already sorted input, reverse-sorted input, missing files and allocation-failure paths.

SOLVED PROBLEM LABORATORY

Twenty-one fully worked programming problems

Each worked problem contains the requirement, reasoning, program, test case and an exam-oriented Malayalam explanation. Programs use standard C11 and explicit error checks wherever input, files or dynamic memory are involved.

Module 1 solved problems — control flow, functions and recursion

Solved Problem 1

Selection structure

Electricity-bill calculation using an else-if ladder

Problem: Calculate the energy charge for the following simplified slabs: first 100 units at ₹1.50, next 100 at ₹2.50 and remaining units at ₹4.00. Reject negative units.

1. Read the unit value and validate it.
2. Apply only the first slab when units are at most 100.
3. For 101–200 units, add the complete first slab and the used part of the second slab.
4. Above 200, add both complete lower slabs and the remaining upper-slab charge.

electricity_bill.c

```
#include <stdio.h>

int main(void)
{
    int units;
    double charge;

    printf("Enter consumed units: ");
    if (scanf("%d", &units) != 1 || units < 0) {
        fprintf(stderr, "Invalid unit value\n");
        return 1;
    }

    if (units <= 100) {
        charge = units * 1.50;
    } else if (units <= 200) {
        charge = 100 * 1.50 + (units - 100) * 2.50;
    } else {
        charge = 100 * 1.50 + 100 * 2.50
                 + (units - 200) * 4.00;
    }

    printf("Energy charge = Rs. %.2f\n", charge);
    return 0;
}
```

```
Input: 250
Energy charge = Rs. 600.00
```

മലയാളം: Slab calculation-ൽ മുഴുവൻ units-നും അവസാന rate multiply ചെയ്യരുത്. ഓരോ slab-ലുള്ള units വേർതിരിച്ചാണ് charge കണക്കാക്കുന്നത്.

Solved Problem 2 switch

Menu-driven arithmetic calculator

Problem: Read two numbers and an operator. Perform addition, subtraction, multiplication or division. Prevent division by zero.

calculator.c

```
#include <stdio.h>

int main(void)
{
    double a, b;
    char operation;

    printf("Enter expression (example 12.5 * 4): ");
    if (scanf("%lf %c %lf", &a, &operation, &b) != 3) {
        fprintf(stderr, "Invalid expression\n");
        return 1;
    }

    switch (operation) {
    case '+': printf("Result = %.2f\n", a + b); break;
    case '-': printf("Result = %.2f\n", a - b); break;
    case '*': printf("Result = %.2f\n", a * b); break;
    case '/':
        if (b == 0.0) {
            fprintf(stderr, "Division by zero is not allowed\n");
            return 1;
        }
        printf("Result = %.2f\n", a / b);
        break;
    default:
        fprintf(stderr, "Unknown operator\n");
        return 1;
    }
    return 0;
}
```

Discrete operator choices-ന് switch അനുയോജ്യമാണ്. Division case-ൽ divisor zero ആണോ പരിശോധിക്കണം.

Solved Problem 3

Loop tracing

Armstrong number for a three-digit value

Problem: Check whether a three-digit integer equals the sum of the cubes of its digits.

armstrong.c

```
#include <stdio.h>

int main(void)
{
    int number, copy, sum = 0;

    printf("Enter a three-digit positive integer: ");
    if (scanf("%d", &number) != 1
        || number < 100 || number > 999) {
        fprintf(stderr, "Expected a three-digit integer\n");
        return 1;
    }

    copy = number;
    while (copy != 0) {
        int digit = copy % 10;
        sum += digit * digit * digit;
        copy /= 10;
    }

    printf("%d is %san Armstrong number\n",
           number, sum == number ? "" : "not ");
    return 0;
}
```

For 153	Digit	Cube	Running sum
1st iteration	3	27	27
2nd iteration	5	125	152
3rd iteration	1	1	153

Solved Problem 4

Function

GCD and LCM using a reusable function

Problem: Find GCD using Euclid’s algorithm and calculate LCM without duplicating logic.

gcd_lcm.c

```
#include <stdio.h>
#include <stdlib.h>

int gcd(int a, int b)
{
    a = abs(a);
    b = abs(b);
    while (b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

int main(void)
{
    int x, y;
    printf("Enter two integers: ");
    if (scanf("%d %d", &x, &y) != 2) {
        return 1;
    }

    int divisor = gcd(x, y);
    long long lcm = (x == 0 || y == 0)
                  ? 0
                  : llabs((long long)x / divisor * y);

    printf("GCD = %d\nLCM = %lld\n", divisor, lcm);
    return 0;
}
```

GCD logic function-ലാക്കിയതിനാൽ അതേ function മറ്റു programs-ലും reuse ചെയ്യാം. LCM multiplication overflow കുറയ്ക്കാൻ division ആദ്യം ചെയ്യുന്നു.

Solved Problem 5

Recursion

Recursive sum of digits

Recursive definition: digitSum(n) = n when n < 10; otherwise n % 10 + digitSum(n / 10).

recursive_digit_sum.c

```c
#include <stdio.h>

unsigned int digit_sum(unsigned int number)
{
    if (number < 10U) {
        return number;
    }
    return number % 10U + digit_sum(number / 10U);
}

int main(void)
{
    unsigned int number;
    if (scanf("%u", &number) != 1) {
        return 1;
    }
    printf("Digit sum = %u\n", digit_sum(number));
    return 0;
}
```

```
digit_sum(5729)
= 9 + digit_sum(572)
= 9 + 2 + digit_sum(57)
= 9 + 2 + 7 + digit_sum(5)
= 23
```

Recursive call-ലേക്ക് പോകുമ്പോൾ number / 10 ഉപയോഗിച്ച് problem size കുറയുന്നു. number < 10 base case ആണ്.

Module 2 solved problems — arrays, search, sort and strings

Solved Problem 6

1D array

Find the largest and second-largest distinct values

second_largest.c

```
#include <limits.h>
#include <stdio.h>

int main(void)
{
    int values[] = {14, 7, 21, 21, 9, 18};
    int size = (int)(sizeof values / sizeof values[0]);
    int largest = INT_MIN;
    int second = INT_MIN;

    for (int i = 0; i < size; i++) {
        if (values[i] > largest) {
            second = largest;
            largest = values[i];
        } else if (values[i] > second && values[i] != largest) {
            second = values[i];
        }
    }

    if (second == INT_MIN) {
        printf("No distinct second-largest value\n");
    } else {
        printf("Largest = %d, second largest = %d\n",
               largest, second);
    }
    return 0;
}
```

Duplicate maximum value second-largest ആയി എടുക്കാതിരിക്കാൻ `values[i] != largest` പരിശോധിക്കുന്നു.

Solved Problem 7 2D array

Matrix multiplication

For A of order $r_1 \times c_1$ and B of order $r_2 \times c_2$, multiplication is possible only when $c_1 = r_2$.

2 × 3 multiplied by 3 × 2

```
#include <stdio.h>

int main(void)
{
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int b[3][2] = {{7, 8}, {9, 10}, {11, 12}};
    int product[2][2] = {{0, 0}, {0, 0}};

    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 2; col++) {
            for (int k = 0; k < 3; k++) {
                product[row][col] += a[row][k] * b[k][col];
            }
        }
    }

    for (int row = 0; row < 2; row++) {
        for (int col = 0; col < 2; col++) {
            printf("%d%c", product[row][col], col == 1 ? '\n' : ' ');
        }
    }
    return 0;
}
```

```
58 64
139 154
```

Solved Problem 8

Linear and binary search

Search comparison with a dry run

Search for 31 in the sorted array {4, 9, 15, 22, 31, 47, 53}.

Binary-search step	low	high	mid	a[mid]	Decision
1	0	6	3	22	Target is larger; low = 4
2	4	6	5	47	Target is smaller; high = 4
3	4	4	4	31	Found at index 4

Both search functions

```
int linear_search(const int a[], int size, int target)
{
    for (int i = 0; i < size; i++) {
        if (a[i] == target) {
            return i;
        }
    }
    return -1;
}

int binary_search(const int a[], int size, int target)
{
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Linear search unsorted data-ലും പ്രവർത്തിക്കും. Binary search വേഗമാണ്, പക്ഷേ data sorted ആയിരിക്കണം.

Solved Problem 9

Selection sort trace

Trace selection sort for 29, 10, 14, 37, 13

Pass	Smallest in unsorted part	Array after swap
Initial	—	29, 10, 14, 37, 13
1	10	10, 29, 14, 37, 13
2	13	10, 13, 14, 37, 29
3	14	10, 13, 14, 37, 29
4	29	10, 13, 14, 29, 37

After pass *i*, position *i* contains the correct smallest remaining element. Time complexity remains $O(n^2)$ even for already

sorted input.

Solved Problem 10

Quicksort partition

Understand one Lomuto partition step

For {8, 3, 1, 7, 0, 10, 2}, choose the final element 2 as pivot. Values less than or equal to 2 move left. After partition, the array becomes {1, 0, 2, 7, 3, 10, 8}; pivot 2 reaches its final index 2. Quicksort then processes the left and right subarrays independently.

Exam point: Explain the base condition `low < high`, partition function, pivot placement and two recursive calls. Average complexity is $O(n \log n)$, while poor pivots can produce $O(n^2)$.

Solved Problem 11

String

Palindrome string ignoring letter case

palindrome.c

```
#include <ctype.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    char text[100];
    if (fgets(text, sizeof text, stdin) == NULL) return 1;
    text[strcspn(text, "\n")] = '\0';

    size_t left = 0;
    size_t right = strlen(text);
    if (right > 0) right--;

    int palindrome = 1;
    while (left < right) {
        if (tolower((unsigned char)text[left])
            != tolower((unsigned char)text[right])) {
            palindrome = 0;
            break;
        }
        left++;
        right--;
    }

    printf("%s\n", palindrome ? "Palindrome" : "Not palindrome");
    return 0;
}
```

Left index മുന്നോട്ടും right index പിന്നോട്ടും നീക്കി corresponding characters compare ചെയ്യുന്നു.

Module 3 solved problems — pointers and dynamic memory

Solved Problem 12

Pointer output parameters

Return quotient and remainder from one function

Multiple outputs through pointers

```
int divide_values(int dividend, int divisor,
                  int *quotient, int *remainder)
{
    if (divisor == 0 || quotient == NULL || remainder == NULL) {
        return 0;
    }
    *quotient = dividend / divisor;
    *remainder = dividend % divisor;
    return 1;
}
```

Call with `divide_values(17, 5, &q, &r)`. The function returns success/failure normally and writes two calculated values through pointers.

Solved Problem 13

Pointer arithmetic

Reverse an array using two pointers

reverse_array.c

```
void reverse(int *first, int *last)
{
    while (first < last) {
        int temporary = *first;
        *first = *last;
        *last = temporary;
        first++;
        last--;
    }
}

/* For int a[5], call reverse(a, a + 4); */
```

Pointers array-ന്റെ രണ്ട് അറ്റങ്ങളിൽ നിന്ന് നടുവിലേക്ക് നീങ്ങുന്നു. Pointer comparison same array-ലുള്ള elements-ക്കിടയിൽ ഉപയോഗിക്കുന്നു.

Solved Problem 14

calloc

Dynamically read marks and calculate average

dynamic_average.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t count;
    printf("Number of students: ");
    if (scanf("%zu", &count) != 1 || count == 0) {
        fprintf(stderr, "Invalid count\n");
        return 1;
    }

    double *marks = calloc(count, sizeof *marks);
    if (marks == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return 1;
    }

    double total = 0.0;
    for (size_t i = 0; i < count; i++) {
        printf("Mark %zu: ", i + 1);
        if (scanf("%lf", &marks[i]) != 1
            || marks[i] < 0.0 || marks[i] > 100.0) {
            fprintf(stderr, "Invalid mark\n");
            free(marks);
            return 1;
        }
        total += marks[i];
    }

    printf("Average = %.2f\n", total / count);
    free(marks);
    return 0;
}
```

Runtime-ൽ count ലഭിക്കുന്നതിനാൽ dynamic allocation ഉപയോഗിക്കുന്നു. Error path-ലും normal path-ലും memory free ചെയ്യണം.

Solved Problem 15

realloc

Grow an integer list without losing the original allocation

Suppose an existing list contains four values and two more values must be added. The safe pattern is:

Safe resize pattern

```
size_t old_count = 4;
size_t new_count = 6;
int *temporary = realloc(values, new_count * sizeof *values);

if (temporary == NULL) {
    /* values still points to the old valid block */
    free(values);
    values = NULL;
    return 1;
}

values = temporary;
values[old_count] = 50;
values[old_count + 1] = 60;
```

Incorrect: `values = realloc(values, ...)`. If allocation fails, NULL overwrites the only address of the original block and causes a memory leak.

Solved Problem 16

Array of pointers

Print subject names in alphabetical order

Sort pointers, not string literals

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    const char *names[] = {"Pointers", "Arrays", "Files", "Functions"};
    int count = (int)(sizeof names / sizeof names[0]);

    for (int i = 0; i < count - 1; i++) {
        for (int j = i + 1; j < count; j++) {
            if (strcmp(names[i], names[j]) > 0) {
                const char *temporary = names[i];
                names[i] = names[j];
                names[j] = temporary;
            }
        }
    }

    for (int i = 0; i < count; i++) puts(names[i]);
    return 0;
}
```

Module 4 solved problems — structures, union, files and command line

Solved Problem 17

Array of structures

Find the highest-scoring student

student_topper.c

```
#include <stdio.h>

struct Student {
    int roll;
    char name[40];
    float mark;
};

int topper_index(const struct Student students[], int count)
{
    if (count <= 0) return -1;
    int best = 0;
    for (int i = 1; i < count; i++) {
        if (students[i].mark > students[best].mark) {
            best = i;
        }
    }
    return best;
}

int main(void)
{
    struct Student students[] = {
        {1, "Anu", 83.5f},
        {2, "Basil", 91.0f},
        {3, "Chitra", 88.5f}
    };
    int count = (int)(sizeof students / sizeof students[0]);
    int best = topper_index(students, count);

    printf("Topper: %s, mark %.1f\n",
           students[best].name, students[best].mark);
    return 0;
}
```

Structure array-യിൽ ഓരോ element ഒരു complete student record ആണ്. Index മാത്രം return ചെയ്യുന്നതിനാൽ caller-ക്ക് മുഴുവൻ record access ചെയ്യാം.

Solved Problem 18

Structure calculation

Employee gross salary

Structure-returning function

```
struct Employee {
    int id;
    char name[40];
    double basic;
};

double gross_salary(const struct Employee *employee)
{
    double hra = employee->basic * 0.20;
    double da = employee->basic * 0.10;
    return employee->basic + hra + da;
}
```

For a basic salary of ₹20,000, HRA = ₹4,000 and DA = ₹2,000, so gross salary = ₹26,000.

Solved Problem 19

Tagged union

Safely store either an integer or real value

Enum tag + union

```
enum DataType { TYPE_INTEGER, TYPE_REAL };

struct DataValue {
    enum DataType type;
    union {
        int integer;
        double real;
    } value;
};

void print_value(const struct DataValue *data)
{
    if (data->type == TYPE_INTEGER) {
        printf("%d\n", data->value.integer);
    } else {
        printf("%.2f\n", data->value.real);
    }
}
```

Enum tag ഏത് union member ഇപ്പോൾ valid ആണെന്ന് സൂചിപ്പിക്കുന്നു. Wrong member read ചെയ്യുന്നത് ഒഴിവാക്കാം.

Count characters, words and lines in a text file

file_statistics.c

```
#include <ctype.h>
#include <stdio.h>

int main(void)
{
    FILE *file = fopen("input.txt", "r");
    if (file == NULL) {
        perror("input.txt");
        return 1;
    }

    long characters = 0;
    long words = 0;
    long lines = 0;
    int previous_space = 1;
    int character;

    while ((character = fgetc(file)) != EOF) {
        characters++;
        if (character == '\n') lines++;

        if (isspace((unsigned char)character)) {
            previous_space = 1;
        } else if (previous_space) {
            words++;
            previous_space = 0;
        }
    }

    if (characters > 0) {
        /* Count a final line even when it has no trailing newline. */
        rewind(file);
        int last = 0;
        while ((character = fgetc(file)) != EOF) last = character;
        if (last != '\n') lines++;
    }

    fclose(file);
    printf("Characters: %ld\nWords: %ld\nLines: %ld\n",
           characters, words, lines);
    return 0;
}
```

Word തുടങ്ങുന്നത് previous character whitespace ആയിരിക്കുമ്പോൾ current character non-whitespace ആകുന്ന സമയത്താണ്.

Solved Problem 21

Command line + files

Copy one file to another using command-line filenames

file_copy.c

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "Usage: %s source destination\n", argv[0]);
        return 1;
    }

    FILE *source = fopen(argv[1], "rb");
    if (source == NULL) {
        perror(argv[1]);
        return 1;
    }

    FILE *destination = fopen(argv[2], "wb");
    if (destination == NULL) {
        perror(argv[2]);
        fclose(source);
        return 1;
    }

    unsigned char buffer[4096];
    size_t count;
    int failed = 0;

    while ((count = fread(buffer, 1, sizeof buffer, source)) > 0) {
        if (fwrite(buffer, 1, count, destination) != count) {
            failed = 1;
            break;
        }
    }
    if (ferror(source)) failed = 1;

    if (fclose(source) != 0) failed = 1;
    if (fclose(destination) != 0) failed = 1;

    if (failed) {
        fprintf(stderr, "File copy failed\n");
        return 1;
    }
    printf("File copied successfully\n");
    return 0;
}
```

```
gcc -std=c11 -Wall -Wextra file_copy.c -o file_copy
./file_copy source.txt backup.txt
```

Source, destination filenames command line-ൽ ലഭിക്കുന്നു. രണ്ട് files-നും separate error handling വേണം.

WORKBOOK

Practice questions with checkpoints

Attempt each task before opening the checkpoint. Modify at least one condition and retest the program.

Workbook 1 — Number classification

Write one program that reports whether an integer is positive/negative/zero, even/odd and divisible by both 3 and 5.

Checkpoint: Use separate decisions because the classifications are independent. The “divisible by both” condition is `number % 3 == 0 && number % 5 == 0`.

Workbook 2 — Recursive power

Implement `power(base, exponent)` for a non-negative integer exponent.

Checkpoint: Base case exponent 0 returns 1. Recursive case returns base × power(base, exponent − 1).

Workbook 3 — Matrix transpose

Read a 2 × 3 matrix and produce a 3 × 2 transpose.

Checkpoint: Assign `transpose[col][row] = original[row][col]`.

Workbook 4 — Binary-search precondition

Modify a program so it refuses binary search when the array is not in ascending order.

Checkpoint: Before searching, loop from index 1 and check whether `a[i] < a[i - 1]`.

Workbook 5 — Implement my_strcmp

Compare two strings without calling `strcmp`.

Checkpoint: Advance while both characters are equal and non-null. Return the difference of the first unequal unsigned characters.

Workbook 6 — Dynamic growing list

Begin with capacity 2. Double capacity with realloc whenever the list becomes full.

Checkpoint: Maintain separate `size` and `capacity`. Use a temporary pointer for realloc.

Workbook 7 — Sort structures

Sort student records in descending order of mark, then ascending name when marks are equal.

Checkpoint: Compare marks first; use `strcmp` only for the tie.

Workbook 8 — Append and search a record file

Append a student record to a text file, then reopen it and search by roll number.

Checkpoint: Use append mode for insertion and read mode for searching. Loop while all expected fields are read successfully.

DEBUGGING HANDBOOK

Common mistakes and corrections

Mistake	Why it fails	Correction
`if (x = 5)`	Assignment, not comparison.	Use `if (x == 5)`.
Missing `&` in numeric scanf	`scanf` needs an address.	`scanf("%d", &x)` and check the return value.
Using integer division accidentally	`sum / count` discards the fraction when both are integers.	Cast one operand: `(double)sum / count`.
Array index equals size	Out-of-bounds access.	Loop while `i < size`.
Binary search on unsorted input	Ordering assumption is violated.	Sort first, verify order or use linear search.
String without `'\0'`	String functions read beyond intended data.	Reserve and preserve terminator space.
Using `gets()`	No buffer-size limit; unsafe.	Use `fgets()`.
Dereferencing NULL/uninitialized pointer	No valid target object.	Initialize, validate, then dereference.
Overwriting a pointer with failed realloc	Original block address is lost.	Use a temporary pointer.
Using memory after free	The object lifetime has ended.	Do not dereference it; set the pointer to NULL when useful.
Reading an inactive union member	Stored representation may not match.	Track the active member with an enum tag.
`while (!feof(file))`	EOF is set only after a failed read.	Loop on the input function's return value.
Ignoring fopen failure	Later operations dereference a NULL file pointer.	Check `file == NULL` immediately.
Forgetting `fclose` or `free`	Resource leak or incomplete output.	Release every successfully acquired resource.

FINAL REVISION

One-page memory map

Module 1

- Program structure
- if/switch/loops
- #include/#define
- Function prototype and call
- Storage classes

- Base and recursive cases

Module 2

- 1D/2D arrays
- Row-major storage
- Linear/binary search
- Selection/quicksort
- Null-terminated strings
- Array parameters

Module 3

- Address and dereference
- Pointer arithmetic
- Output parameters
- malloc/calloc
- Safe realloc/free
- Arrays and pointers

Module 4

- Structures and arrays
- Dot and arrow
- Union and enum
- File modes
- Sequential I/O
- argc/argv

Malayalam last-day cue: Control flow → Functions → Recursion → Arrays → Search/Sort → Strings → Pointers → Dynamic memory → Structures → Union/Enum → Files → Command line എന്ന ക്രമത്തിൽ revise ചെയ്യുക. ഓരോ topic-നും കുറഞ്ഞത് ഒരു program dry run ചെയ്യുക.

Likely one-mark areas

- Define recursion.
- Purpose of a function prototype.
- Meaning of static local variable.
- Requirement for binary search.
- String terminator.

- Meaning of NULL.
- Difference between dot and arrow.
- Purpose of free().

Likely short answers

- Storage classes.
- Selection versus repetition.
- Linear versus binary search.
- Selection sort versus quicksort.
- Array–pointer relationship.
- malloc versus calloc.
- Structure versus union.
- File modes.

Likely programs

- Recursive factorial/GCD.
- Matrix multiplication.
- Search and sort.
- String functions.
- Dynamic array and realloc.
- Student structure.
- Sequential file processing.
- Command-line file copy.

PRACTICE MODEL QUESTION PAPER

PROGRAMMING IN C • Maximum Marks: 75 • Time: 3 Hours

This original practice paper follows the uploaded syllabus coverage. It is not an official board question paper. A complete master answer guide is provided below.

PART A — Answer all questions. Each carries 1 mark. [5 × 1 = 5]

1. What is the purpose of a function prototype?
2. State the essential requirement for binary search.
3. What does the null character do in a C string?
4. Name the function used to release dynamically allocated memory.

5. What is the difference between `.` and `->` in structure access?

PART B — Answer any five. Each carries 8 marks. [5 × 8 = 40]

1. Explain control structures with suitable C fragments.
2. Explain storage classes, scope, lifetime and visibility.
3. Write and explain a recursive program to find factorial.
4. Compare linear and binary search and write a binary-search function.
5. Explain selection sort and quicksort.
6. Explain strings in C and write functions for length and comparison.
7. Explain pointer arithmetic and dynamic memory allocation.
8. Explain structures, unions and enumerations with examples.

PART C — Answer any two. Each carries 15 marks. [2 × 15 = 30]

1. Develop a menu-driven array program supporting input, display, linear search, selection sort and average.
2. Explain pointers and write programs for swap, dynamic array allocation and two-dimensional array access using pointer notation.
3. Develop a student-record program using an array of structures and sequential file storage. Include proper file-error handling.

MASTER ANSWER KEY

Complete model answers

Use these answers to understand structure and marking points. In the examination, explain the logic in your own words and keep programs properly indented.

Part A answers

Q	Model answer
1	A function prototype declares the function name, return type and parameter types before the function is called, enabling compile-time type checking.
2	The array must be sorted in the same order assumed by the binary-search algorithm.
3	The null character `'\0'` marks the end of a C string.
4	`free()`.
5	The dot operator accesses a member through a structure object; the arrow operator accesses a member through a pointer to a structure.

B1 — Control structures

Control structures determine the order in which statements execute.

Type	Purpose	Examples
Sequence	Statements run one after another.	Input → calculation → output
Selection	Selects one path according to a condition.	if, if-else, else-if, switch
Repetition	Repeats a block.	for, while, do-while
Unconditional transfer	Changes normal flow directly.	break, continue, goto, return

Representative fragments

```
if (mark >= 50) {
    printf("Pass\n");
} else {
    printf("Fail\n");
}

for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}

switch (choice) {
case 1: printf("Add\n"); break;
case 2: printf("Search\n"); break;
default: printf("Invalid\n");
}
```

Condition true/false അനുസരിച്ച് selection path മാറുന്നു. Loop condition false ആകുന്നതുവരെ repetition നടക്കും.

B2 — Storage classes, scope, lifetime and visibility

Scope specifies where a name can be used. **Lifetime** specifies how long the object exists. **Visibility** is whether a declaration can be seen at a particular program point.

Class	Default use	Lifetime	Important feature
auto	Local variables	During block execution	Default local storage class
register	Frequently accessed local variable	During block execution	Compiler optimisation hint
static local	Local variable retaining state	Entire program	Block scope, persistent value
static file-scope	Private global object/function	Entire program	Internal linkage
extern	Object defined in another location/file	Entire program	External declaration

Static local example

```
void counter(void)
{
    static int calls = 0;
    calls++;
    printf("%d\n", calls);
}
```

B3 — Recursive factorial

Factorial is defined as $0! = 1$ and $n! = n \times (n - 1)!$ for $n > 0$. The first rule is the base case; the second is the recursive case.

Complete answer

```
#include <stdio.h>

unsigned long long factorial(unsigned int n)
{
    if (n <= 1U) {
        return 1ULL;
    }
    return n * factorial(n - 1U);
}

int main(void)
{
    unsigned int n;
    printf("Enter n (0 to 20): ");
    if (scanf("%u", &n) != 1 || n > 20U) {
        fprintf(stderr, "Invalid input\n");
        return 1;
    }
    printf("%u! = %llu\n", n, factorial(n));
    return 0;
}
```

Trace for 4!: factorial(4) = 4 × factorial(3) = 4 × 3 × factorial(2) = 4 × 3 × 2 × factorial(1) = 24.

B4 — Linear search versus binary search

Point	Linear search	Binary search
Data order	Any order	Must be sorted
Method	Checks elements one by one	Repeatedly halves the search range
Worst time	O(n)	O(log n)
Suitable for	Small or unsorted data	Large sorted arrays

Binary-search function

```
int binary_search(const int a[], int size, int target)
{
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

B5 — Selection sort and quicksort

Selection sort: In each pass, find the smallest element in the unsorted part and swap it with the first unsorted position. It is in-place and $O(n^2)$.

Quicksort: Choose a pivot, partition the array into values not greater than the pivot and values greater than it, place the pivot in its final position, and recursively sort both parts. Average time is $O(n \log n)$; worst time is $O(n^2)$.

Selection-sort core

```
for (int i = 0; i < size - 1; i++) {
    int min_index = i;
    for (int j = i + 1; j < size; j++) {
        if (a[j] < a[min_index]) min_index = j;
    }
    int temporary = a[i];
    a[i] = a[min_index];
    a[min_index] = temporary;
}
```

B6 — Strings, length and comparison

A C string is an array of characters terminated by `'\0'`. Input must not exceed the destination buffer. Standard operations include length, copy, concatenation, comparison and substring searching.

Custom length and comparison

```
#include <stddef.h>

size_t my_strlen(const char text[])
{
    size_t length = 0;
    while (text[length] != '\0') length++;
    return length;
}

int my_strcmp(const char left[], const char right[])
{
    size_t i = 0;
    while (left[i] != '\0' && left[i] == right[i]) i++;
    return (unsigned char)left[i] - (unsigned char)right[i];
}
```

A zero comparison result means equal strings; a negative or positive result indicates lexical order.

B7 — Pointer arithmetic and dynamic memory

A pointer stores an address. `&x` gives the address of x and `*p` accesses the pointed object. Adding one to an int pointer advances by `sizeof(int)` bytes to the next int object.

Function	Use
malloc	Allocate uninitialised storage
calloc	Allocate an array and zero-initialise its bytes
realloc	Resize an existing allocation
free	Release allocated storage

Allocation pattern

```
int *values = malloc((size_t)count * sizeof *values);
if (values == NULL) {
    return 1;
}
/* use values */
free(values);
values = NULL;
```

B8 — Structure, union and enumeration

A structure allocates storage for every member and groups different data types into one record. A union shares one storage area among all members. An enumeration creates named integer constants.

Example

```
enum ValueType { INTEGER_VALUE, REAL_VALUE };

struct Student {
    int roll;
    char name[40];
    float mark;
};

struct Value {
    enum ValueType type;
    union {
        int integer;
        double real;
    } data;
};
```

Use `student.mark` for an object and `student_pointer->mark` for a pointer. The enum tag records which union member is active.

Part C full solutions

C1 — Complete menu-driven array program

menu_array.c

```
#include <stdio.h>

#define CAPACITY 100

void read_array(int a[], int *size)
{
    int requested;
    printf("Number of elements (1-%d): ", CAPACITY);
    if (scanf("%d", &requested) != 1
        || requested < 1 || requested > CAPACITY) {
        printf("Invalid size\n");
        *size = 0;
        return;
    }

    for (int i = 0; i < requested; i++) {
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("Invalid input\n");
            *size = 0;
            return;
        }
    }
    *size = requested;
}

void display_array(const int a[], int size)
{
    if (size == 0) {
        printf("Array is empty\n");
        return;
    }
    for (int i = 0; i < size; i++) {
        printf("%d%c", a[i], i == size - 1 ? '\n' : ' ');
    }
}

int linear_search(const int a[], int size, int target)
{
    for (int i = 0; i < size; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}

void selection_sort(int a[], int size)
{
    for (int i = 0; i < size - 1; i++) {
        int min_index = i;
```

```
        for (int j = i + 1; j < size; j++) {
            if (a[j] < a[min_index]) min_index = j;
        }
        int temporary = a[i];
        a[i] = a[min_index];
        a[min_index] = temporary;
    }
}

double average(const int a[], int size)
{
    long long total = 0;
    for (int i = 0; i < size; i++) total += a[i];
    return size > 0 ? (double)total / size : 0.0;
}

int main(void)
{
    int values[CAPACITY];
    int size = 0;
    int choice;

    do {
        printf("\n1 Read\n2 Display\n3 Search\n");
        printf("4 Selection sort\n5 Average\n0 Exit\nChoice: ");
        if (scanf("%d", &choice) != 1) return 1;

        switch (choice) {
        case 1:
            read_array(values, &size);
            break;
        case 2:
            display_array(values, size);
            break;
        case 3: {
            int target;
            printf("Target: ");
            if (scanf("%d", &target) != 1) return 1;
            int index = linear_search(values, size, target);
            if (index >= 0) printf("Found at index %d\n", index);
            else printf("Not found\n");
            break;
        }
        case 4:
            selection_sort(values, size);
            printf("Array sorted\n");
            break;
        case 5:
            if (size == 0) printf("Array is empty\n");
```

```
            else printf("Average = %.2f\n", average(values, size));
            break;
        case 0:
            printf("Exiting\n");
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 0);

    return 0;
}
```

Program functions ആയി വിഭജിച്ചിരിക്കുന്നു. Size validation, empty-array check, search not-found case എന്നിവ mark ലഭിക്കാൻ സഹായിക്കുന്ന പ്രധാന points ആണ്.

C2 — Pointer explanation and complete program set

Definition: A pointer is a variable containing the address of an object. Pointer parameters allow modification of caller data. Pointer arithmetic traverses arrays. Dynamic allocation creates objects whose required size is known only at run time.

(a) Swap

swap.c

```c
void swap(int *left, int *right)
{
    int temporary = *left;
    *left = *right;
    *right = temporary;
}

/* int x = 10, y = 20; swap(&x, &y); */
```

(b) Dynamic array

dynamic_array.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count;
    if (scanf("%d", &count) != 1 || count <= 0) return 1;

    int *values = malloc((size_t)count * sizeof *values);
    if (values == NULL) return 1;

    long long total = 0;
    for (int i = 0; i < count; i++) {
        if (scanf("%d", &values[i]) != 1) {
            free(values);
            return 1;
        }
        total += values[i];
    }

    printf("Average = %.2f\n", (double)total / count);
    free(values);
    return 0;
}
```


(c) Two-dimensional array through pointer notation

Equivalent notation

```
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

for (int row = 0; row < 2; row++) {
    for (int col = 0; col < 3; col++) {
        printf("%d ", *(*(matrix + row) + col));
    }
    putchar('\n');
}
```

`matrix + row` points to a row; dereferencing it gives that row, adding col moves to the required element, and the final dereference reads the integer.

C3 — Student structures with sequential file storage

student_file.c

```
#include <stdio.h>

#define MAX_STUDENTS 50

struct Student {
    int roll;
    char name[40];
    float mark;
};

int read_students(struct Student students[], int capacity)
{
    int count;
    printf("Number of students: ");
    if (scanf("%d", &count) != 1
        || count < 1 || count > capacity) {
        return 0;
    }

    for (int i = 0; i < count; i++) {
        printf("Roll name mark: ");
        if (scanf("%d %39s %f",
                  &students[i].roll,
                  students[i].name,
                  &students[i].mark) != 3) {
            return 0;
        }
    }
    return count;
}

int save_students(const char *filename,
                  const struct Student students[], int count)
{
    FILE *file = fopen(filename, "w");
    if (file == NULL) {
        perror(filename);
        return 0;
    }

    for (int i = 0; i < count; i++) {
        if (fprintf(file, "%d %s %.2f\n",
                    students[i].roll,
                    students[i].name,
                    students[i].mark) < 0) {
            fclose(file);
            return 0;
        }
    }
```

```
    return fclose(file) == 0;
}

void display_file(const char *filename)
{
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        perror(filename);
        return;
    }

    struct Student student;
    printf("\nStored records\n");
    while (fscanf(file, "%d %39s %f",
                  &student.roll,
                  student.name,
                  &student.mark) == 3) {
        printf("%d %-15s %.2f\n",
               student.roll, student.name, student.mark);
    }
    fclose(file);
}

int main(void)
{
    struct Student students[MAX_STUDENTS];
    int count = read_students(students, MAX_STUDENTS);
    if (count == 0) {
        fprintf(stderr, "Invalid student data\n");
        return 1;
    }

    if (!save_students("students.txt", students, count)) {
        fprintf(stderr, "Could not save records\n");
        return 1;
    }

    display_file("students.txt");
    return 0;
}
```

The solution demonstrates structure definition, array of structures, structure members, passing an array to functions, text-file writing, text-file reading, checking `fopen`, validating formatted input and closing every opened file.

Twenty important questions and answers

1. Why does array indexing start at zero?

Because `a[i]` is defined as `*(a+i)`; index zero means zero element offsets from the base address.

2. Declaration versus definition of a function?

A declaration states the interface. A definition provides the executable body.

3. Why does recursion need a base case?

Without a base case, calls continue until stack memory is exhausted.

4. What is a function's activation record?

It is the stack information for one function call, commonly including parameters, local variables, return address and saved execution state.

5. Why must binary search use sorted data?

Its decision to discard the left or right half depends on order.

6. What is row-major order?

Elements of one row of a multidimensional array are stored contiguously before the next row.

7. Why does a string need one extra character position?

The extra position stores the terminating null character.

8. What is pointer arithmetic scaled by?

By the size of the pointed type. Adding one moves to the next object of that type.

9. What is a dangling pointer?

A pointer referring to an object whose lifetime has ended, for example memory that has already been freed.

10. malloc versus calloc?

`malloc` allocates uninitialised storage; `calloc` allocates an array and zero-initialises its bytes.

11. Why use a temporary pointer with realloc?

On failure, realloc returns NULL while the old block remains valid. Direct assignment could lose the old address.

12. What is a memory leak?

Allocated memory that is no longer reachable and therefore cannot be released or reused by the program.

13. Structure versus union?

A structure allocates storage for all members; union members share the same storage.

14. What does the arrow operator mean?

`pointer->member` is shorthand for `(*pointer).member`.

15. Why combine enum with union?

The enum records which shared union member currently contains a valid value.

16. Why is while(!feof(file)) incorrect?

EOF becomes true only after a read attempts to go past the end. Test the read function's return value instead.

17. Difference between w and a file modes?

Write mode truncates existing content; append mode preserves it and writes new data at the end.

18. What does perror do?

It prints a supplied message followed by the system description of the most recent library/system error.

19. What are argc and argv?

`argc` is the count of command-line strings; `argv` is the array of pointers to those strings.

20. Why is argv[0] useful?

It normally contains the program invocation name and can be displayed in a usage message.

REFERENCES FROM THE SYLLABUS

Books and online learning resources

Code	Book / Resource
T1	E. Balagurusamy, *Programming in ANSI C*, 7th Edition.
R1	Yashavant Kanetkar, *Let Us C*.
R2	Paul J. Deitel and Harvey Deitel, *C How to Program*.
R3	Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition.
R4	Herbert Schildt, *C: The Complete Reference*.
R5	Byron Gottfried, *Schaum's Outline of Programming with C*.
Online	NPTEL course 106104128; Programiz C Programming; TutorialsPoint C Programming.